Investigating Simplicity Bias in Neural Networks via Iterative Subspace Isolation (ISI) Loops

Rudransh Khandelwal
BYTE Coding Society, Maharaja Agrasen Institute of Technology (MAIT)

I. Abstract

This paper investigates the idea: "Simplicity Bias in neural networks" : the tendency for models to rely on easy shortcuts rather than complex, robust shapes. We demonstrate this bias by training networks on the MNIST dataset with injected shortcuts (a corner square and background colors). When these shortcuts are removed during testing, standard models' accuracy fails drastically, going below to 90.20% and even up to 10.02%. These experiments concluded that whenever a simpler feature was available to the neural network it would shift to that and create what many call a "Simplicity Bias". To test the assumption that neural networks assign heavier weight to these simpler features than complex features, we introduce the Iterative Subspace Isolation (ISI) loop. Our algorithm identifies, freezes, and suppresses the dominant weights, forcing the network to train its lower-magnitude weights (which according to the assumptions contain the complex weights) on the actual digit shapes. Our tests proved that networks never truly lose the ability to learn complex shapes but just focus more on simpler ideas. However, breaking this "Simplicity Bias" requires a two-step strategy: a strong initial cut to disrupt the dominance of simpler features, followed by a relaxed window to refine and polish the fine details. Using this approach, we brought the network's accuracy all the way back up to 99.62% and 99.60%, proving that simplicity bias is just a temporary training hurdle rather than a permanent limitation.

II. Introduction

A. What is the Simplicity Bias?

Deep neural networks possess immense computational power, allowing them to map out complex functions. However, standard training procedures consistently run into a major roadblock known as Simplicity Bias. When multiple features can be used to solve a task, neural networks naturally lean towards learning the simpler, easy-to-read features over more complex, robust ones. This bias often causes a massive drop in the network's overall classification accuracy once those simple shortcuts are missing from the testing data.

B. What is this Proposal about?

This research proposal details an empirical study conducted as part of the "Falsification Proof" challenge organized by my college coding society, BYTE at Maharaja Agrasen Institute of Technology (MAIT). As a student who has just entered the field of artificial intelligence, my objective was not only to study research papers and test them, but also to try and create a experimental demonstration that can help me to further explore this field and Learn a lot of things. While my baseline setups confirmed that standard networks blindly prioritize simple data shortcuts, I developed an intervention mechanism to bypass this limitation: the Iterative Subspace Isolation (ISI) loop. This custom algorithm systematically dampens the

internal parameters dedicated to lazy shortcuts, forcing the network's hidden layers to distribute their weight capacity toward robust structural features.

C. My proposal in a Nutshell

I put my framework to the test by using both a self-built single-neuron model and a 256-neuron Multi-Layer Perceptron (MLP) architecture trained on the MNIST dataset. By injecting explicit shortcuts—ranging from localized 2×2 pixel corner squares to multi-class global background brightness intensities—I captured severe simplicity traps. In my most extreme background trap, the standard network lost all shape recognition ability, crashing down to a random-guessing accuracy floor of 10.02%. By applying my self developed ISI loop, we forced the network to see past the background noise and read the actual handwritten lines, driving the final validation accuracy up to an elite ceiling of 99.60%. This paper outlines our complete theoretical reasoning, step-by-step algorithmic mechanics, and empirical verification logs, confirming that simplicity bias is a temporary optimization hurdle that can be successfully overcome.

D. What are ISI Loops and how does they help to Prove the Hypothesis?

ISI loops stands for Iterative Subspace Isolation loops.

- **Iterative :** The algorithm runs in multiple sequential cycles (3 loops) to continuously refine the weights.
- **Subspace :** It divides the network's parameters into distinct mathematical zones—separating the simple shortcut weights from the complex structural weights.
- **Isolation :** It freezes and dampens the lazy shortcut connections so the network is forced to learn real digit shapes using its other parameters.

I independently created this kind of Loops, so that i can force the model to train on complex features instead of being trained on the simpler features. If after training the network on these loops the accuracy goes up, I can confidently say that the neural network choose to bent towards a simpler feature despite the fact that it could have been focused more on complex features for a better accuracy.

E. Did I used LLMs and AI Models?

To ensure full transparency, I completely agree to the fact that i have used LLMs and AI models as my mentor and code writers. While I designed the entire logic, parameter ordering rules, step-by-step matrix masking flows, and decay schedules, the physical Python code files in this repository were generated with the assistance of AI due to my current stage of programming fluency. Every single script has been thoroughly mapped, tested, and verified to ensure it executes my exact structural design, establishing a reliable workflow combining my thinking ability with AI's super fast code generating skills

III. Research Question & Falsifiable Hypotheses

My main research question is:
Do neural networks have the capacity to learn complex features but still they prefer to learn simple features?.

To scientifically test this question, we define the following formal hypotheses:

- **My Hypothesis (H_1):** If simpler features exist in the training data then the neural network will set higher weights on the shortcut-tracking parameters creating a Simplicity Bias ultimately lowering the accuracy when tested on data without those simpler features. But by using a multi phase isolation loop, we can make the network shift the focus of the weights from simpler to complex ones, yielding measurable performance increases on clean adversarial test sets where the simpler features are completely removed.
- **Null Hypothesis (H_0):** Neural networks naturally balance both simple and complex features equally during standard training; my ISI intervention will fail to yield an accuracy increase because the complex shapes were already optimized normally.

Our framework establishes two primary postulates to evaluate H_1:

1) **Postulate 1:** After applying ISI Loops on the simplicity biased neural network, the weights will be more focused on complex features and a significant increase in accuracy shall be visible.
2) **Postulate 2:** The optimal weight reduction factor (γ) must scale upward across training iterations to prevent the weights of complex features from dropping down till the same extent as the simpler ones

If H_1 is true than the above mentioned postulates should be true. Hence in this experiment, i tried to prove whether the postulates are true or not ultimately proving/contradicting H_1.

IV. Algorithmic Framework: Iterative Subspace Isolation (ISI)

To test my hypotheses, I introduce the Iterative Subspace Isolation (ISI) loop algorithm. The core assumption of our algorithm states that if simplicity bias causes a network to depend on a shortcut, the largest absolute weight magnitudes must belong to those shortcut features. Therefore, identifying, freezing, retraining and reducing these dominant pathways will force the remaining open units (which contains the complex features) to look deeper into the image canvas. This is would ultimately train the network on complex features which would boost it's accuracy.

A. The Mathematical Formulation

Let $\mathbf{W} \in \mathbb{R}^{m \times n}$ represent a parameter weight matrix in a given network layer. The absolute modulus matrix is defined as $|\mathbf{W}|$, where each element is the absolute value of the corresponding weight.

We define a binary gradient freeze tensor $\mathcal{M} \in \{0, 1\}^{m \times n}$ based on a targeted threshold percentile P (the hyperparameter). The elements of the tensor are computed as:

$$\mathcal{M}_{ij} = \begin{cases} 0 & \text{if } |\mathbf{W}_{ij}| \geq \text{Percentile}(|\mathbf{W}|, P) \\ 1 & \text{if } |\mathbf{W}_{ij}| < \text{Percentile}(|\mathbf{W}|, P) \end{cases} \tag{1}$$

During the next training step, we modify how the weights are updated. We separate the locked shortcut connections and shrink their strength using a reduction factor $\gamma \in [0, 1]$. The parameters are adjusted dynamically according to:

$$\mathbf{W}_{\text{new}} = (\mathcal{M} \odot \mathbf{W}_{\text{old}}) + \gamma \cdot ((\mathbf{1} - \mathcal{M}) \odot \mathbf{W}_{\text{old}}) \tag{2}$$

where $\odot$ denotes the element-wise multiplication. Before moving lets see, how this equation works.

To visualize this coordinate splitting mechanism clearly, the step-by-step element-wise matrix pipeline can be represented as follows:

Step A: Un-frozen Structural Pathway ($\mathcal{M} \odot \mathbf{W}_{\text{old}}$)

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \odot \begin{bmatrix} 1.5 & 0.8 & 9.4 \\ 2.1 & 8.8 & 0.4 \end{bmatrix} = \begin{bmatrix} 1.5 & 0.8 & 0.0 \\ 2.1 & 0.0 & 0.4 \end{bmatrix}$$

Step B: Isolated Shortcut Pathway with Penalty ($\gamma \cdot [(\mathbf{1} - \mathcal{M}) \odot \mathbf{W}_{\text{old}}]$)

$$0.10 \cdot \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \odot \begin{bmatrix} 1.5 & 0.8 & 9.4 \\ 2.1 & 8.8 & 0.4 \end{bmatrix} \right) = \begin{bmatrix} 0.0 & 0.0 & 0.94 \\ 0.0 & 0.88 & 0.0 \end{bmatrix}$$

Step C: Combined Final Matrix ($\mathbf{W}_{\text{new}}$)

$$\mathbf{W}_{\text{new}} = \begin{bmatrix} 1.5 & 0.8 & 0.0 \\ 2.1 & 0.0 & 0.4 \end{bmatrix} + \begin{bmatrix} 0.0 & 0.0 & 0.94 \\ 0.0 & 0.88 & 0.0 \end{bmatrix} = \begin{bmatrix} 1.5 & 0.8 & 0.94 \\ 2.1 & 0.88 & 0.4 \end{bmatrix}$$

As this mathematical flow demonstrates, the complex shape pathways retain their original full strength, while the dominant simplicity shortcuts are successfully choked down to just 10% of their original impact.

What this basically does is, it identifies the highest weighted neurons, freezes them and reduces them to 10% of their original value. While the un-freezed neurons are sent back to train again on the data set where the model cant do any change to the freezed neurons.

During the backward learning pass, the learning signals (gradients) are filtered using our binary mask $\mathcal{M}$. This safely restricts any weight adjustments exclusively to the open, structural weights while blocking changes to the frozen parameters:

$$\nabla_{\mathbf{W}} \mathcal{L}_{\text{masked}} = \mathcal{M} \odot \nabla_{\mathbf{W}} \mathcal{L} \tag{3}$$

To implement this framework over multiple training cycles, we construct a step-by-step list of reduction factors $\mathbf{\Gamma} = [\gamma_1, \gamma_2, \ldots, \gamma_K]$ across K sequential cycles. This allows our weight multiplier to scale dynamically and shift smoothly over time.

B. Algorithmic Flow

The complete training pipeline runs through three distinct operational phases:

1) **Phase 1: Shortcut Trap Phase:** The model is trained normally on data that contains an easy shortcut (like a corner block or background color). Because networks are lazy, simplicity bias naturally forces the early weights to focus entirely on this shortcut.
2) **Phase 2: Identification and Masking Phase:** The algorithm sorts all the network weights from largest to smallest. It applies our binary mask $\mathcal{M}$ to lock down the top-percentile weights, assuming these are the ones tracking the shortcut coordinates.
3) **Phase 3: Forced Shape Retraining Phase:** The frozen shortcut weights are scaled down using our multiplier γ_k. This breaks their dominance and forces the network's remaining open weights to focus on learning the actual, complex digit shapes.

C. The Mathematics in a Nutshell

1) Train the model standardly on the dataset where simplicity bias is suspected to occur.
2) Sort the trained parameter weights based on their absolute magnitude to identify which specific connections are the highest contributors to the model's outputs.
3) Freeze the top-percentile weights, as these dominant pathways are responsible for tracking the simpler shortcut features according to our core assumption.
4) Retrain the model on the same data with those shortcut weights frozen, forcing the optimization pass to distribute features across the open, complex parameter paths.
5) Apply a localized reduction factor to those frozen parameters and recombine both weight partitions together, yielding a retrained matrix where complex shape features are sharpened and shortcut impacts are reduced.
6) Repeat this intervention loop 2–3 times while scaling up the reduction factor, ensuring that as complex features naturally migrate into higher weight tiers, they are safely preserved from parameter capacity starvation.

D. How my Postulates proves the claim?

My postulates depend on the assumption that neural network prioritizes Simpler/Shortcut features over complex features. Using that assumption I made ISI Loops which can reduce the simplicity bias of the network and make the network train itself on complex features.

Now, because ISI loops forces the model to train itself on complex features; the Accuracy must increase. And in every iteration as the top contributing weights will be shifted from shortcut ones to complex ones; the reduction factor also must increase to retain the trained complex weights.

Note : *You should note the point that reduction factor is not directly proportional to the reducing of the freezed weights but is indirectly due to the equation :*

$$(\mathbf{W}_{\text{frozen}})_{\text{new}} = \gamma \cdot [(\mathbf{1} - \mathcal{M}) \odot (\mathbf{W}_{\text{frozen}})_{\text{old}}] \tag{4}$$

V. Experimental Design & Controls

I designed simple dataset traps using MNIST to check how my algorithm works across two distinct model architectures.

A. Model Architectures

I used two control environments to analyze architectural scaling dynamics:

- **Single-Neuron Baseline Model ('neuron_basecode.py', 'neuron_experiment.py'):** A single-neuron network trained using a sigmoid activation function to perform binary classification (determining whether an input image represents the digit "0" or not).
- **Multi-Neuron MLP Matrix Network ('mlp_matrix_experiment.py'):** A Multi-Layer Perceptron scaling up to a 256-neuron hidden layer utilizing ReLU activations, designed to perform full 10-class digit classification.

B. Dataset Synthesis and Shortcut Injections

To simulate the Simplicity Bias, I made two separate shortcut traps:

- **Localized Corner Shortcut:** I modified the 60,000 MNIST training images by injecting a 2×2 pixel white square into the top-left corner of every image of the number "0". The testing set remains completely clean, meaning all image corners are naturally black.
- **Global Background Context Trap:** I altered the global background pixels of the training images, assigning a unique background intensity to each digit class. Same as before, the validation test set backgrounds are kept perfectly black.

C. Mathematical Proof of the Trapped Statistical Floors

After I have modified the dataset, I had to check the drop in accuracy and more importantly, the reason behind that drop.

1) *Single-Neuron Floor (90.20%)*

The MNIST validation dataset consists of exactly 10,000 images. Within this distribution, there are exactly 9,020 images representing digits 1 through 9 (the "Not 0" class) and exactly 980 images representing the true digit "0". Because the standard single neuron becomes completely dependent on the easy 2×2 white corner block during Phase 1, it learns a cheat code: "If the corner coordinates contain a white patch, classify as 0; if the corner is black, classify as Not 0."

When this trained model is evaluated on the clean test data where all image corners are naturally black, the network classifies every single input image as "Not 0". As a result, it misclassifies all 980 true zeros, but achieves perfect accuracy on the 9,020 non-zero digits. The resulting accuracy is verified exactly as:

$$\text{Test Accuracy} = \frac{9,020 \text{ (Correct Non-Zeros)} + 0 \text{ (Correct Zeros)}}{10,000 \text{ (Total Test Images)}} \times 100 = 90.20\% \tag{5}$$

The model achieves a high-looking accuracy just by identifying every single as "Not 0", without every recognizing a single 0 Image.

2) *Multi-Neuron MLP Floor (10.02%)*

When scaling the experiment to the 10-class background color setup, the standard network loses all shape comprehension and latches entirely onto the background color intensity. When evaluated on black backgrounds, it defaults to a uniform random guessing state across the 10 distinct classes, yielding a test accuracy of exactly **10.02%**—the mathematical expression of approximately 1-out-of-10 random guessing probability ($\approx 10\%$).

VI. Impact of ISI Loops

After the accuracy on the neural networks showed a subsequent drop, I re-trained the shortcut-biased neural networks on my ISI Loops, and here is the impact of them on the networks.

1) **Single-Neuron Neural Network:** This model showed a significant impact. After just one iteration, the accuracy sky-rocketed to 96.3%, and after 3 iterations the accuracy settled at 97.82%. This was before I tested the perfect reduction factor combo for the 3 loops. Once I found the best combination, i.e., $[0.1, 0.9, 0.9]$, the accuracy got a huge jump and settled at 98.72%.

2) **Multi-Neuron MLP Network:** Furthermore, when I applied the ISI Loops to the MLP model facing the corner block trap, the accuracy jumped from 90.20% all the way to 99.62%. Then, when it was applied to the model which was trained on the background color trap, the accuracy got a huge jump from 10.02% up to 99.60%.

These both were ground-breaking results for me, as I have improved the accuracy of the models by a good factor, and both of my postulates were proven to be correct. Ultimately, acknowledging my assumption.

VII. Experimental Verification Logs & Quantitative Proof

I systematically executed my proposed experiment suite to gather empirical data. The results of these trials are logged comprehensively in Table I, providing direct verification of our postulates.

TABLE I
EXPERIMENTAL RESULTS MATRIX

Seq.	Evaluation Test Condition	Detailed Parameter Configuration	Observed Acc.	Shift	Status
1	Baseline Trap Verification	Adding 2×2 pixel white corner block	$99.25\% \rightarrow 90.20\%$	-9.05%	Passed
2a	Initial ISI Validation (1 Loop)	Single cycle retraining, unfrozen subspace	$90.20\% \rightarrow 90.90\%$	$+0.70\%$	Passed
2b	Normalized Static Schedule	3 Cycles, static $\Gamma = [0.95, 0.95, 0.95]$ reduction	$90.20\% \rightarrow 90.20\%$	$\pm 0.00\%$	Failed
2c	Aggressive Static Schedule	3 Cycles, static $\Gamma = [0.40, 0.40, 0.40]$ reduction	$90.20\% \rightarrow 97.82\%$	$+7.62\%$	Passed
2d	Static Parameter Sweep	Optimal single-cycle static $\gamma = 0.10$ execution	$90.20\% \rightarrow 98.72\%$	$+8.52\%$	Passed
2e	Hard Erasure Schedule	3 Cycles, absolute 100% reduction $\Gamma = [0.10, 0.00, 0.00]$	$90.20\% \rightarrow 96.23\%$	$+6.03\%$	Passed
2f	Hyperparameter Sweep Run	16-Variant Sweep, optimal $\Gamma = [0.10, 0.90, 0.90]$	$90.20\% \rightarrow 98.67\%$	$+8.47\%$	Passed
3a	Scaled 256-Neuron MLP Matrix	Intersecting Layer Decoupling (W_1 and W_2 masked)	$90.20\% \rightarrow 99.62\%$	$+9.42\%$	Passed
3b	Global Context Rescued MLP	Regional Context Masking on Background Trap	$10.02\% \rightarrow 99.60\%$	$+89.58\%$	Passed

Note: *Here Passed and Failed represents if the result followed my result or not.*

A. Analysis of Postulate 1: Evidence of Structural Recovery

The data from my tests strongly supports Postulate 1. When I applied the ISI Loop to the single-neuron system with the best static factor, the clean test accuracy jumped from 90.20% to 98.72% (an 8.52% gain).

When I scaled this up to the 256-neuron MLP architecture, the model achieved almost perfect feature recovery, climbing all the way to 99.62%. This is just 0.08% away from a completely untrapped baseline model (99.70%). Similarly, the global background context rescue brought the model back up to 99.60% from the 10.02% random guessing floor (a massive +89.58% net gain). This provides solid proof that the complex features are never truly deleted during early training. They just sit quietly inside the lower-magnitude weights, waiting to be unlocked by my algorithm.

B. Analysis of Postulate 2: Verification of the Stepped Schedule

My automated sweep logs (`hyperparameter_sweep.py`) prove Postulate 2 by showing what happens when we change the reduction schedule. When I tested a hard erasure schedule where the factor drops straight to zero ($\mathbf{\Gamma} = [0.1, 0.0, 0.0]$), the network dropped in accuracy during later epochs, falling from an intermediate high of 98.85% down to 96.23%.

This happens because a single neuron has very tight memory capacity, so completely zeroing out the weights deletes shared connections that map both the shortcut borders and the actual digit lines at the same time. On the other hand, starting with a strong penalty followed by a relaxed preservation window ($\mathbf{\Gamma} = [0.1, 0.9, 0.9]$) hit the highest single-neuron score of 98.67%. This proves that breaking simplicity bias requires a two-step strategy: an initially small factor to break the lazy shortcut habit, followed by a larger factor to give the network room to polish fine handwriting updates.

VIII. Deep Analytical Insights & Alternative Explanations

Based on my experimental findings, I can share a few important insights into how a network updates its internal weights when dealing with shortcuts:

- **The Optimization Pathing Hurdle:** My study challenges the common idea that simplicity bias is a permanent structural flaw in neural networks. Instead, I argue it is just a shortcut habit driven by early training paths. Localized corner squares or solid backgrounds are simple, low-frequency patterns that require much shorter learning paths to drop the early loss, compared to the complex curves of handwritten numbers. Standard SGD simply takes the path of least resistance during the very first epoch. Therefore, simplicity bias is a consequence of early training choices rather than a fixed limitation of the model's architecture.
- **The Intersecting Layer Bottleneck:** During my multi-layer MLP tests, I found that freezing weights only in the first layer matrix (W_1) did not work; the output layer matrix (W_2) stayed completely filled with the pre-trained shortcut habits, keeping my clean accuracy stuck at the 90.20% floor. Symmetrically applying gradient freeze masks across both connected matrices (W_1 and W_2) at the same time is mechanically necessary to completely break the shortcut pipeline and force learning signals backward into the open hidden units.
- **Regional Context Isolation:** In larger multi-neuron networks, sorting weights purely by their global size can cause small, localized shortcut paths to get drowned out by the background noise of the surrounding active pixels. Restricting the sorting loop to look closely at targeted spatial areas provides the precise resolution needed to isolate and fix hidden units that have fallen into the shortcut trap.

IX. Limitations & Future Research Scalability

While my algorithm achieves complete feature recovery on flat MLP models, I acknowledge a few important limitations in its current design:

1) **Architectural Scalability:** My experiments were conducted on simple single-neuron and single-hidden-layer MLP setups using synthetic shortcuts. This baseline setup does not automatically guarantee that the same isolation loop can scale easily to deep, 50-layer Convolutional Neural Networks (CNNs) or Vision Transformers (ViTs), where shortcut paths and complex geometric shapes are deeply tangled together.
2) **Prior Shortcut Knowledge:** Right now, my ISI loop relies on knowing the shortcut coordinates beforehand to mask the target paths. In real-world problems, shortcuts are often hidden and unknown. Future iterations should connect this framework with dynamic tracking tools—like Integrated Gradients or Grad-CAM maps—to automatically find shortcut coordinates before triggering the isolation mask.
3) **Correlation Strength Boundaries:** In this study, I kept the shortcut-to-label correlation strength locked at a perfect 100% during the training phase. Future research should test lower correlation thresholds to discover the exact boundary where standard training loops naturally look past shortcuts on their own without needing layer interventions.

X. Conclusion

This research proves that neural networks definitely have a simplicity bias, but it only happens when these easy shortcuts exist in the training data. It works exactly like an electrical circuit: if there is a low-resistance shortcut path available, almost all the current will flow directly through it, leaving the high-resistance almost path empty. Standard gradient descent does the exact same thing by rushing down the easiest learning route. By proving that my ISI loop can block that shortcut and bring shape accuracy all the way back up from a 10.02% floor to an elite 99.62% ceiling, I have demonstrated that simplicity bias is just a temporary pathing hurdle rather than a permanent architectural limit. My findings show that the network is fully capable of learning complex shapes, but we must manually break that lazy shortcut circuit to unlock the hidden structural features waiting in the lower weights.

References

1) BYTE MAIT Challenge Brief (2026): *The Principle of Appropriate Complexity and the Falsification Sprint Parameters*.
2) Shah, H., Tamuly, K., Raghunathan, A., Jain, P., & Netrapalli, P. (2020): “The Pitfalls of Simplicity Bias in Neural Networks.” *Advances in Neural Information Processing Systems (NeurIPS 2020)*, 33, 9573-9585.
3) Morwani, M., Podder, S., & Sharma, S. (2023): “Simplicity Bias in 1-Hidden Layer Neural Networks.” *Advances in Neural Information Processing Systems (NeurIPS 2023)*.
4) Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, X., Hamprecht, F. A., Bengio, Y., & Courville, A. (2019): “On the Spectral Bias of Neural Networks.” *International Conference on Machine Learning (ICML 2019)*, PMLR, 5301-5310.